

Remove Duplicates from Sorted Array

O(N) Time

O(1) Space

Two Pointers · In-place Deduplication · Striver TUF+ Revision Notes

PROBLEM STATEMENT

Given an integer array `nums` sorted in non-decreasing order, remove the duplicates **in-place** so each unique element appears only once, keeping the relative order. Return `k`, the number of unique elements — only the first `k` slots of `nums` need to hold the answer; whatever sits beyond them does not matter.

Example: `[1,1,2,2,2,3,3] → k = 3`, `nums` starts with `[1,2,3, ...]`

Approach 1 · Brute Force (Using a Set)

Intuition

The naive idea is to reach for a data structure that simply refuses to store duplicates — a set. Push every element of the array into the set; repeats are silently dropped. Since `std::set` also keeps its contents in sorted order, copying the set back into the front of the array gives exactly the sorted, duplicate-free result we want.

Approach

- Declare a `set<int> s` and traverse `nums`, inserting every element into it.
- Store the set's size in `k` — that is the count of unique elements.
- Copy all elements of the set back into `nums` starting from index 0, then return `k`.

KEY OBSERVATION

This works even if the array were *not* sorted, because `std::set` sorts for us. That extra power is exactly what we are over-paying for — the input is already sorted, so the sorting work of the set is wasted effort.

Code (C++)

```
int removeDuplicates(vector<int>& nums) {  
    // set stores unique elements in sorted order  
    set<int> s;  
  
    for (int val : nums) {  
        s.insert(val);           // duplicates are ignored  
    }  
  
    int k = s.size();           // number of unique elements  
  
    int j = 0;  
    for (int val : s) {  
        nums[j++] = val;        // copy unique values back  
    }  
  
    return k;  
}
```

Dry Run · `nums = [1,1,2,2,2,3,3]`

☒ First occurrence (kept) ☐ Duplicate (ignored) ☐ Not scanned yet **val ↓ current element**

Iteration · scanning `nums`, inserting each value into the set

i=0	val ↓ 1 1 2 2 2 3 3	<code>nums[0]=1: 1 is new -> inserted</code> <code>set = {1}</code>
i=1	val ↓ 1 1 2 2 2 3 3	<code>nums[1]=1: 1 already in set -> ignored</code> <code>set = {1}</code>
i=2	val ↓ 1 1 2 2 2 3 3	<code>nums[2]=2: 2 is new -> inserted</code> <code>set = {1,2}</code>
i=5	val ↓ 1 1 2 2 2 3 3	<code>nums[5]=3: 3 is new -> inserted</code> <code>set = {1,2,3}</code>
i=6	val ↓ 1 1 2 2 2 3 3	<code>nums[6]=3: 3 already in set -> ignored (scan done)</code> <code>set = {1,2,3}</code>

Scan complete — set holds only unique values (sorted)

set



set = {1, 2, 3} → k = 3

Copy set back into nums[0..k-1] — final result

nums



nums[0..2] = 1, 2, 3 (the answer). Return k = 3. Positions from index 3 onward are leftover junk and are ignored.

Complexity

Time: $O(N \log N) + O(N)$

Space: $O(N)$

- Inserting N elements into a set costs $O(\log N)$ each (it is a balanced BST), so building the set is $O(N \log N)$. Copying the unique values back is one more pass, $O(N)$. Total $O(N \log N) + O(N)$.
- In the worst case every element is unique, so the set holds N values → $O(N)$ auxiliary space. This breaks the "in-place" requirement of the problem.

Approach 2 · Optimal (Two Pointers, In-place)

Intuition

Think of a bookshelf of alphabetically sorted books where some titles repeat. Walk along the shelf: keep the first book, then look at the next one. If it matches the last book you kept, skip it. If it is different, slide it up right next to the last kept book. When you reach the end, the front of the shelf holds every unique title once, in order — and the leftovers at the back simply do not matter.

The array being **sorted** is what makes this safe: all copies of a value sit next to each other, so comparing against just the last kept element is enough to detect a duplicate.

Approach

- Set $i = 0$ (position of the last unique element found) and run j from 1 to the end.
- If $\text{nums}[i] == \text{nums}[j]$: it is a duplicate — do nothing, let j advance.
- If $\text{nums}[i] != \text{nums}[j]$: a new unique value is found → increment i , then set $\text{nums}[i] = \text{nums}[j]$.
- After the loop, $i + 1$ is the count of unique elements. Return it.

INVARIANT

At every step, $\text{nums}[0..i]$ holds all unique values seen so far, in their original sorted order. That is why one pass is enough and no extra space is needed.

Code (C++)

```
int removeDuplicates(vector<int>& nums) {
    int i = 0; // last unique element's position

    for (int j = 1; j < nums.size(); j++) {
        // array is sorted, so a different value must be new
        if (nums[i] != nums[j]) {
            i++; // make room for it
            nums[i] = nums[j]; // place the unique value
        }
        // equal -> duplicate, just let j move on
    }

    return i + 1; // count of unique elements
}
```

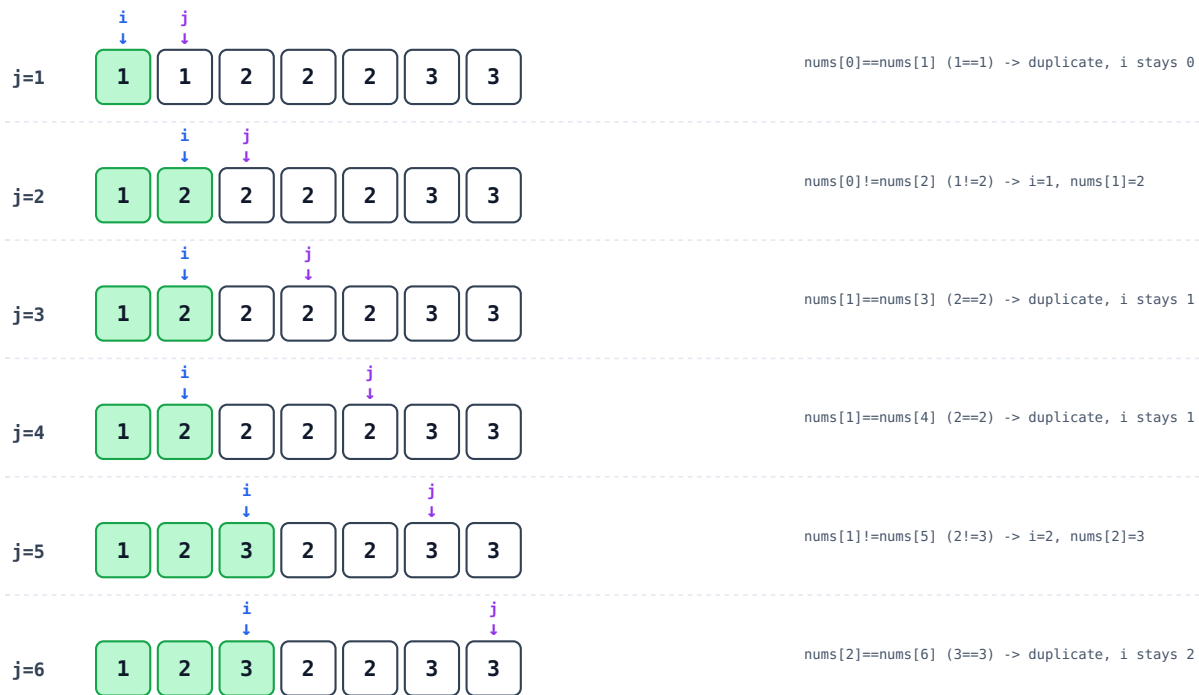
Dry Run · nums = [1,1,2,2,2,3,3]

☒ Unique region [0..i] ☐ Not finalized i ↓ last unique j ↓ scanner

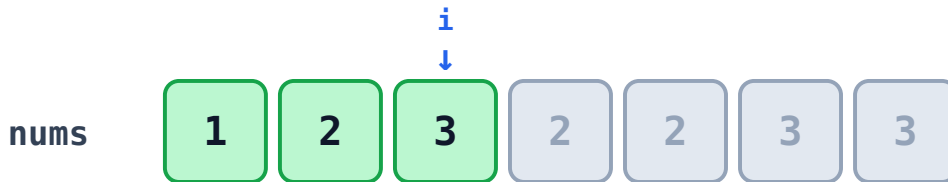
Initial state (i = 0, j starts at 1)

nums





Loop ends (`i = 2`) — final result



`nums[0..2] = 1, 2, 3` are the unique elements. Return `i + 1 = 3`. The tail (greyed) is ignored by the caller.

Complexity

Time: **$O(N)$**

Space: **$O(1)$**

- `j` runs from 1 to `N-1` exactly once and each step does $O(1)$ work (one comparison, at most one increment plus one assignment). No nested loops, no sorting $\rightarrow O(N)$.
- Only the two integer variables `i` and `j` are used and all writes happen inside the given array $\rightarrow O(1)$ auxiliary space, which is what makes this the truly in-place solution.

Common Revision Section

Corner Case (both approaches)	What Happens
Empty array	Set: set stays empty, <code>k = 0</code> . Optimal: loop never runs, returns <code>i + 1 = 1</code> — guard with <code>if (nums.empty()) return 0;</code> if the problem allows an empty input.
Single element	Both return 1. The optimal loop body never executes (<code>j</code> starts at 1), so <code>nums</code> is untouched — correct.
All elements identical e.g. <code>[2,2,2,2]</code>	Set: holds one value, <code>k = 1</code> . Optimal: no comparison ever differs, <code>i</code> stays 0, returns 1. Both correct.
No duplicates at all e.g. <code>[1,2,3,4]</code>	Every comparison differs, so <code>i</code> advances every step and each element is copied onto itself. Returns <code>N</code> — array unchanged.
Two elements, equal / different	<code>[5,5] → returns 1</code> ; <code>[5,9] → returns 2</code> . Smallest case where the loop actually runs.
Negative numbers	Handled by both — only equality/inequality is used, never the sign or magnitude.
Unsorted input	Not valid for this problem. The optimal approach would silently produce a wrong answer, since it only compares against the last kept element.

Pattern Used

Two Pointers — slow/fast in-place partition (write pointer + read pointer). j reads every element; i marks the boundary of the finalized region and only moves when something worth keeping is found. Recognize it whenever a problem says "*modify the array in-place, return the new length*" and asks to remove, filter, or compact elements while preserving order. **Similar:** Move Zeroes, Remove Element (LC 27), Remove Duplicates from Sorted Array II (keep at most twice), Remove Duplicates from Sorted List, Sort Colors.

Tips to Remember

- Return $i + 1$, **not** i — i is an index, the answer is a count.
- Start j at 1, not 0. Starting at 0 compares `nums[0]` with itself and breaks the logic.
- Mistake:** comparing `nums[j]` with `nums[j - 1]` instead of `nums[i]`. It happens to work here, but the `nums[i]` form is the one that generalises to the "keep at most K copies" variants.
- Mistake:** incrementing i before writing, or writing before incrementing — the order is $i++$ *then* `nums[i] = nums[j]`.
- Don't waste time "cleaning" the tail of the array. The problem only cares about the first k slots.
- Rule of thumb:** sorted input + "remove/compact in-place + return length" → two pointers, one writing and one reading.

Quick Revision (30-60 sec)

PROBLEM

Sorted array → remove duplicates in-place, return count of unique elements.

PATTERN

Two Pointers — write pointer i , read pointer j .

KEY IDEA

Sorted ⇒ duplicates are adjacent. If `nums[i] != nums[j]`: $i++$, `nums[i] = nums[j]`.

COMPLEXITY

Optimal $O(N)/O(1)$. Set $O(N \log N)/O(N)$.

RETURN

$i + 1$ (count, not index).

EDGE CASE

Empty array — $i + 1$ wrongly gives 1, so guard it.

Approach Comparison

	Main Idea	Time	Space	When to Use
Brute · Set	Insert everything into a set (drops duplicates, keeps sorted order), copy back, return size.	$O(N \log N)$ + $O(N)$	$O(N)$	Only as a first idea, or when the input is <i>not</i> sorted and extra space is acceptable.
Optimal · Two Pointers	Single pass; i marks the last unique element, j scans ahead and copies new values forward.	$O(N)$	$O(1)$	Always preferred here — exploits the sorted input, truly in-place, expected in interviews.